

Security Audit Report

MBR Login Customiser — WordPress Plugin

Audited version	2.0.0	Report date	3 July 2026
Scope of this review	2.0.0 additions	Author	Robert Palmer
Vendor	Little Web Shack / Made by Robert	Distribution	Self-hosted (GPL)

Executive summary

Version 2.0.0 adds three features to MBR Login Customiser: passkey (WebAuthn / FIDO2) sign-in, security alerts, and trusted devices. This review covers the new modules and the small changes made to the existing two-factor and passkey gates to support them. The findings and controls from the 1.9.0 audit remain in force; the four code-level items raised there stay resolved as of 1.9.1.

No critical, high, or medium-severity vulnerabilities were identified in the 2.0.0 additions. The passkey implementation performs every server-side check the WebAuthn assertion model requires: a single-use, server-issued challenge; verification of the client-data type and origin; the relying-party ID hash; the user-presence flag; a monotonic signature counter for cloned-authenticator detection; and the assertion signature itself, verified with the platform's own OpenSSL and libsodium extensions rather than any third-party cryptography library. Passwordless mode is strictly additive to the existing password form, so it cannot deny legitimate access.

Five findings were raised, all in the low, informational, or by-design band. They concern the accepted trade-off of a “remember this device” feature, the handling of an administrator-supplied webhook URL, the presence of IP addresses in notifications, a reliability fix to challenge storage, and the bounding of alert volume. None represents an authentication bypass.

Findings at a glance

ID	Finding	Severity	Status
F-08	Trusted device skips the second factor if its cookie is stolen	Low	By design / documented
F-09	Alert webhook is an administrator-supplied URL (SSRF surface)	Info	Accepted / hardening
F-10	IP addresses appear in alert notifications	Info	By design / documented
F-11	Passkey registration challenge storage (object-cache reliability)	Info	Resolved 2.0.0
F-12	Alert volume during sustained attacks	Info	Mitigated (cooldowns)

Scope and methodology

The review was a manual, white-box source read of the three new modules under includes/ (the passkey / WebAuthn module, the alerts module, and the trusted-devices module), the client-side passkey ceremony script, and the two-line additions to the existing two-factor and passkey gates that consult the trusted-device filter. Attention was focused on the areas where each feature could fail:

- **Passkey verification** — challenge issuance and single use, origin and relying-party checks, the user-presence flag, signature-counter handling, and the signature-verification path itself.
- **Credential handling** — how COSE public keys are decoded, stored, and reloaded, and whether a stored key round-trips without corruption.
- **Trusted-device cookies** — signing, user binding, expiry, and server-side revocation.
- **Alert delivery** — the webhook request, the data placed in notifications, and volume control.
- **Request integrity** — nonce coverage on the registration endpoints, and the reasoning for the unauthenticated login endpoints (which are gated by the cryptographic challenge-response rather than a nonce).

Behavioural verification was performed with a headless PHP harness on PHP 8.3 with the OpenSSL and libsodium extensions. It simulated an authenticator for the ES256, RS256, and EdDSA (Ed25519) algorithms and asserted the full path for each: registration parsing, storage and reload of the COSE key, and assertion verification, together with rejection of tampered payloads and enforcement of the signature-counter rule. The trusted-device cookie was checked for valid, wrong-user, forged-signature, expired, and server-side-revoked cases, and the alert logic for its threshold, per-type cooldown, and dual-channel delivery.

Independence note

The code under review was authored during the same engagement as this audit. The review was conducted critically and supported by automated verification, but it is not a substitute for an independent third-party assessment, and should not be read as one.

Detailed findings

F-08 Trusted device skips the second factor if its cookie is stolen

Low · By design / documented

Observation. The trusted-devices feature lets a user skip the second factor on a browser they have marked as trusted. That trust is carried by a cookie, so anyone in possession of the cookie skips the second step on that user's account.

Risk. Low, and inherent to any “remember this device” feature. The cookie only removes the second factor — a valid password or passkey is still required — so a stolen cookie alone does not grant access. Theft of the cookie is broadly equivalent to theft of a live session cookie.

Mitigations. The cookie is HMAC-signed with a stored, plugin-owned secret and cannot be forged; it is bound to the user ID; it carries a hard expiry (30 days by default); and a matching record is held server-side so any device can be revoked from the user's profile, which invalidates the cookie immediately. It is issued only over the login POST when the user opts in, and set HttpOnly, Secure, and SameSite=Lax. The feature is off by default and its trade-off is stated in the settings and user documentation.

F-09 Alert webhook is an administrator-supplied URL

Informational · Accepted / hardening

Observation. When a webhook is configured, the plugin makes a server-side POST to that URL on each alert. A configured URL that resolves to an internal address is a server-side request forgery surface.

Assessment. The URL can only be set by a user with the `manage_options` capability — an administrator, who already has extensive server-side capability through the normal WordPress surface, so the feature grants no new privilege. The request is validated with the platform URL validator, is a non-blocking POST with a short timeout and a fixed JSON body, and its response is neither read nor returned to the browser, which limits its usefulness as an SSRF primitive.

Hardening. Sites with a strict internal-network posture may wish to constrain outbound requests, or an optional host allow-list could be added in a future release.

F-10 IP addresses appear in alert notifications

Informational · By design / documented

Observation. Alert emails and webhook messages include the relevant IP address so the recipient can act on the event.

Assessment. An IP address is personal data in some jurisdictions. Its inclusion is operationally necessary for a security notification and it is sent only to the site's own configured administrator or webhook. The known-IP list retained per administrator for new-location detection is stored as a keyed hash, not in the clear, and is capped. Site owners should account for the notification channel in their own privacy documentation.

F-11 Passkey registration challenge storage

Informational · Resolved 2.0.0

Observation. The registration challenge was first held in a transient. On hosting with a persistent object cache, a transient holding a raw binary value could be dropped between the two ceremony requests, causing registration to fail with a timeout even when completed promptly.

Resolution. The registration challenge is now stored in user meta as a `base64url` (ASCII) string with a short, server-enforced expiry, and cleared on first use. This removes the dependency on transient / object-cache behaviour and preserves single-use, anti-replay handling. The login challenge, which has no authenticated user to key on, remains a transient but is likewise stored as ASCII rather than raw binary. This is a reliability fix with no reduction in the anti-replay property.

F-12 Alert volume during sustained attacks

Informational · Mitigated

Observation. Security events can occur in rapid succession during an attack; naive alerting would generate a large volume of email or webhook traffic.

Mitigation. Each alert type has a configurable cooldown, and failed-login spikes are counted in a rolling window and reported once per cooldown rather than per attempt. This bounds outbound volume while still surfacing the event.

Controls confirmed — passkey verification

The following server-side checks were confirmed present in the assertion and registration paths and exercised by the test harness:

- The authentication challenge is generated on the server, is single-use, and is short-lived, defeating replay of a captured assertion.
- The client-data type and the origin are checked against the site's own origin.

- The relying-party ID hash in the authenticator data is checked against the hash of the site's host.
- The user-presence flag is required.
- The assertion signature is verified with the platform's OpenSSL (ES256, RS256) and libsodium (Ed25519) extensions; no third-party cryptography library is introduced.
- The signature counter is enforced to be non-decreasing where the authenticator supplies one, detecting a cloned credential.
- A duplicate credential identifier is rejected across all users at registration.
- Passwordless mode adds a button beside the password form and never removes the password path, so it cannot cause lockout; a wp-config constant disables the feature site-wide, and an administrator can remove any user's passkeys.

This report covers the 2.0.0 additions and should be read alongside the 1.9.0 audit for the full plugin. It reflects the state of the code at the report date and is provided without warranty.